how can we also integrate physical models of star formation using Kennicutt-Schmidt law directly into the feature engineering?
2/2

ChatGPT said:
Reasoned for 7 seconds

Integrating the Kennicutt–Schmidt law into your feature engineering means incorporating physical expectations about star formation directly into your machine learning inputs. The law relates the star formation rate surface density (Σ_SFR) to the gas surface density (Σ_gas) by a power law (typically, Σ_SFR ∝ Σ_gas^n, with n ≈ 1.4).

Here are some ways to incorporate this physical model:

1. **Create a Predicted SFR Feature:**
   If you have a proxy or measurement for gas surface density (or a related quantity like CO or HI flux), you can compute an expected SFR:
   - For example, using a normalized constant A, compute:
     expected_SFR = A * (gas_density)^1.4
   - This feature can be added as an input so the model learns deviations from the physical law.
2. **Residual Feature:**
   If your dataset includes an observed SFR and you compute an expected SFR from the Kennicutt–Schmidt law, you can create a residual feature:
   - **residual_SFR = observed_SFR - expected_SFR**
     This may capture additional physics or environmental effects not explained by the simple law.
3. **Interaction Terms:**
   You could include interaction terms between gas-related proxies and other features (like metallicity or stellar mass) that might affect the star formation efficiency. For example, if you have gas density and stellar mass, you might compute an interaction like:
   - **gas_density × stellar_mass**
     This can help the model learn complex, non-linear relationships consistent with physical processes.
4. **Normalization and Scaling:**
   The Kennicutt–Schmidt law is often expressed in surface densities. Ensure that your gas density proxy is appropriately normalized (e.g., in units of M⊙/pc²) before applying

the power law.

Example Implementation

Assume you have a column named `gas_density` in your dataset that serves as a proxy for Σ_gas. You could do the following:

python
CopyEdit

```
# Define a normalization constant A based on literature or calibration
A = 1e-4  # This is an example value; you'll need to calibrate this
based on your data

# Compute the expected SFR using the Kennicutt-Schmidt law
df['expected_SFR'] = A * (df['gas_density'] ** 1.4)

# Create a residual feature, comparing observed SFR (if available) to
expected SFR
df['residual_SFR'] = df['observed_SFR'] - df['expected_SFR']

# Now include these new features in your feature set for the model
X = df[['ra', 'dec', 'redshift', 'expected_SFR', 'residual_SFR']]
y = df['observed_SFR']  # or whichever target SFR measurement you're
using
```

If you don't have a direct gas density measurement, you might use another proxy that correlates with gas content (e.g., certain color indices or stellar mass surface density) and derive a similar feature.

Further Refinement

- **Calibration:**
  Calibrate the normalization constant A and the exponent (commonly around 1.4) using a subset of data where the SFR and gas density are well measured.

- **Model Evaluation:**
  Use techniques like SHAP to evaluate how much the new features (expected_SFR and

residual_SFR) contribute to your model's performance, and adjust as needed.

- **Incorporate as Part of an Ensemble:**
  You can also train models with and without these physical features and use ensemble methods to combine predictions.

By integrating these physical models into your feature engineering, your model can leverage domain knowledge to potentially improve its predictions and provide a more interpretable link between astrophysical theory and observational data.

Let me know if you'd like more details on any step or further assistance with the implementation!

You said:

can you adapt the script i'm providing in this message to the recommendations outlined in your previous message? as well as add these adaptations? can you adapt the script i'm providing in this message to the recommendations outlined in your previous message? import numpy as np import pandas as pd from sklearn.model_selection import train_test_split, RandomizedSearchCV, GridSearchCV from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor from sklearn.preprocessing import StandardScaler from sklearn.metrics import mean_squared_error from sklearn.impute import SimpleImputer from scipy.stats import randint, uniform import matplotlib.pyplot as plt import psutil # For monitoring system resources # Step 1: Monitor System Resources (CPU and Memory) def check_system_resources(): memory = psutil.virtual_memory() cpu = psutil.cpu_percent(interval=1) print(f"CPU Usage: {cpu}%") print(f"Memory Usage: {memory.percent}%") return cpu, memory.percent # Step 2: Load the SDSS dataset df = pd.read_csv('SDSSDR18_200000.csv', low_memory=False) # Inspect the first few rows of the dataset print(df.head()) # Step 3: Data Preprocessing # Handle missing values by imputing with the mean imputer = SimpleImputer(strategy='mean') df[['ra', 'dec', 'redshift', 'u', 'g', 'r', 'i', 'z']] = imputer.fit_transform(df[['ra', 'dec', 'redshift', 'u', 'g', 'r', 'i', 'z']]) # Step 4: Feature Engineering # Calculate color indices as additional features df['u_g'] = df['u'] - df['g'] df['g_r'] = df['g'] - df['r'] df['r_i'] = df['r'] - df['i'] df['i_z'] = df['i'] - df['z'] # Select features and target variable X = df[['ra', 'dec', 'redshift', 'u_g', 'g_r', 'r_i', 'i_z']] y = df['redshift'] # Assuming we want to predict redshift; adjust as needed # Step 5: Split data into training and testing sets X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42) # Step 6: Standardize the features scaler = StandardScaler() X_train_scaled = scaler.fit_transform(X_train) X_test_scaled = scaler.transform(X_test) # Step 7: Initialize models rf_model = RandomForestRegressor(random_state=42) gb_model = GradientBoostingRegressor(random_state=42) # Step 8: Define hyperparameter grids for RandomizedSearchCV and GridSearchCV # Randomized Search Hyperparameter Grid rf_param_dist = { 'n_estimators': randint(100, 300), 'max_depth': randint(10, 30), 'min_samples_split': randint(2, 10), 'min_samples_leaf': randint(1, 4), 'max_features': ['sqrt', 'log2', None] } gb_param_dist = { 'n_estimators': randint(100, 300), 'learning_rate': uniform(0.01,

0.3), 'max_depth': randint(3, 7), 'min_samples_split': randint(2, 10), 'min_samples_leaf': randint(1, 4) } # Grid Search Hyperparameter Grid (fine-tuning after RandomizedSearchCV) rf_param_grid = { 'n_estimators': [200, 250], 'max_depth': [20, 25], 'min_samples_split': [2, 5], 'min_samples_leaf': [1, 2], 'max_features': ['sqrt', 'log2'] } gb_param_grid = { 'n_estimators': [200, 250], 'learning_rate': [0.05, 0.1], 'max_depth': [5, 7], 'min_samples_split': [2, 5], 'min_samples_leaf': [1, 2] } # Step 9: Perform RandomizedSearchCV to get a rough idea of the best hyperparameters rf_random_search = RandomizedSearchCV(estimator=rf_model, param_distributions=rf_param_dist, n_iter=10, cv=3, n_jobs=-1, verbose=2, random_state=42) gb_random_search = RandomizedSearchCV(estimator=gb_model, param_distributions=gb_param_dist, n_iter=10, cv=3, n_jobs=-1, verbose=2, random_state=42) # Fit the models using RandomizedSearchCV print("Starting RandomizedSearchCV for RandomForest...") rf_random_search.fit(X_train_scaled, y_train) print("Starting RandomizedSearchCV for GradientBoosting...") gb_random_search.fit(X_train_scaled, y_train) # Step 10: Use GridSearchCV for fine-tuning based on best parameters from RandomizedSearchCV print("Starting GridSearchCV for RandomForest...") rf_best_params = rf_random_search.best_params_ # Get best params from RandomizedSearchCV rf_grid_search = GridSearchCV(estimator=rf_model, param_grid=rf_param_grid, cv=3, n_jobs=-1, verbose=2) rf_grid_search.fit(X_train_scaled, y_train) print("Starting GridSearchCV for GradientBoosting...") gb_best_params = gb_random_search.best_params_ # Get best params from RandomizedSearchCV gb_grid_search = GridSearchCV(estimator=gb_model, param_grid=gb_param_grid, cv=3, n_jobs=-1, verbose=2) gb_grid_search.fit(X_train_scaled, y_train) # Step 11: Evaluate the models rf_best_model = rf_grid_search.best_estimator_ gb_best_model = gb_grid_search.best_estimator_ # Predict on the test set y_pred_rf = rf_best_model.predict(X_test_scaled) y_pred_gb = gb_best_model.predict(X_test_scaled) # Calculate performance metrics mse_rf = mean_squared_error(y_test, y_pred_rf) r2_rf = rf_best_model.score(X_test_scaled, y_test) mse_gb = mean_squared_error(y_test, y_pred_gb) r2_gb = gb_best_model.score(X_test_scaled, y_test) print(f"Random Forest - Mean Squared Error: {mse_rf}, R²: {r2_rf}") print(f"Gradient Boosting - Mean Squared Error: {mse_gb}, R²: {r2_gb}") # Step 12: Optional: Plot feature importances for Random Forest importances = rf_best_model.feature_importances_ indices = np.argsort(importances)[::-1] features = X.columns plt.figure(figsize=(10, 6)) plt.title('Random Forest Feature Importances') plt.bar(range(X.shape[1]), importances[indices], align='center') plt.xticks(range(X.shape[1]), features[indices], rotation=90) plt.xlim([-1, X.shape[1]]) plt.show() # Final system resource check after fitting models cpu, memory = check_system_resources() Script import numpy as np import pandas as pd from sklearn.model_selection import train_test_split, RandomizedSearchCV, GridSearchCV from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor from sklearn.preprocessing import StandardScaler from sklearn.metrics import mean_squared_error from sklearn.impute import SimpleImputer from scipy.stats import randint, uniform import matplotlib.pyplot as plt import psutil # For monitoring system resources # Step 1: Monitor System Resources (CPU and Memory) def check_system_resources(): memory = psutil.virtual_memory() cpu = psutil.cpu_percent(interval=1) print(f"CPU Usage: {cpu}%") print(f"Memory Usage: {memory.percent}%") return cpu, memory.percent # Step 2: Load the SDSS dataset df = pd.read_csv('SDSSDR18_200000.csv', low_memory=False) # Inspect the first few rows of the

dataset print(df.head()) # Step 3: Data Preprocessing # Handle missing values by imputing with the mean imputer = SimpleImputer(strategy='mean') df[['ra', 'dec', 'redshift', 'u', 'g', 'r', 'i', 'z']] = imputer.fit_transform(df[['ra', 'dec', 'redshift', 'u', 'g', 'r', 'i', 'z']]) # Step 4: Feature Engineering # Calculate color indices as additional features df['u_g'] = df['u'] - df['g'] df['g_r'] = df['g'] - df['r'] df['r_i'] = df['r'] - df['i'] df['i_z'] = df['i'] - df['z'] # Select features and target variable X = df[['ra', 'dec', 'redshift', 'u_g', 'g_r', 'r_i', 'i_z']] y = df['redshift'] # Assuming we want to predict redshift; adjust as needed # Step 5: Split data into training and testing sets X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42) # Step 6: Standardize the features scaler = StandardScaler() X_train_scaled = scaler.fit_transform(X_train) X_test_scaled = scaler.transform(X_test) # Step 7: Initialize models rf_model = RandomForestRegressor(random_state=42) gb_model = GradientBoostingRegressor(random_state=42) # Step 8: Define hyperparameter grids for RandomizedSearchCV and GridSearchCV # Randomized Search Hyperparameter Grid rf_param_dist = { 'n_estimators': randint(100, 300), 'max_depth': randint(10, 30), 'min_samples_split': randint(2, 10), 'min_samples_leaf': randint(1, 4), 'max_features': ['sqrt', 'log2', None] } gb_param_dist = { 'n_estimators': randint(100, 300), 'learning_rate': uniform(0.01, 0.3), 'max_depth': randint(3, 7), 'min_samples_split': randint(2, 10), 'min_samples_leaf': randint(1, 4) } # Grid Search Hyperparameter Grid (fine-tuning after RandomizedSearchCV) rf_param_grid = { 'n_estimators': [200, 250], 'max_depth': [20, 25], 'min_samples_split': [2, 5], 'min_samples_leaf': [1, 2], 'max_features': ['sqrt', 'log2'] } gb_param_grid = { 'n_estimators': [200, 250], 'learning_rate': [0.05, 0.1], 'max_depth': [5, 7], 'min_samples_split': [2, 5], 'min_samples_leaf': [1, 2] } # Step 9: Perform RandomizedSearchCV to get a rough idea of the best hyperparameters rf_random_search = RandomizedSearchCV(estimator=rf_model, param_distributions=rf_param_dist, n_iter=10, cv=3, n_jobs=-1, verbose=2, random_state=42) gb_random_search = RandomizedSearchCV(estimator=gb_model, param_distributions=gb_param_dist, n_iter=10, cv=3, n_jobs=-1, verbose=2, random_state=42) # Fit the models using RandomizedSearchCV print("Starting RandomizedSearchCV for RandomForest...") rf_random_search.fit(X_train_scaled, y_train) print("Starting RandomizedSearchCV for GradientBoosting...") gb_random_search.fit(X_train_scaled, y_train) # Step 10: Use GridSearchCV for fine-tuning based on best parameters from RandomizedSearchCV print("Starting GridSearchCV for RandomForest...") rf_best_params = rf_random_search.best_params_ # Get best params from RandomizedSearchCV rf_grid_search = GridSearchCV(estimator=rf_model, param_grid=rf_param_grid, cv=3, n_jobs=-1, verbose=2) rf_grid_search.fit(X_train_scaled, y_train) print("Starting GridSearchCV for GradientBoosting...") gb_best_params = gb_random_search.best_params_ # Get best params from RandomizedSearchCV gb_grid_search = GridSearchCV(estimator=gb_model, param_grid=gb_param_grid, cv=3, n_jobs=-1, verbose=2) gb_grid_search.fit(X_train_scaled, y_train) # Step 11: Evaluate the models rf_best_model = rf_grid_search.best_estimator_ gb_best_model = gb_grid_search.best_estimator_ # Predict on the test set y_pred_rf = rf_best_model.predict(X_test_scaled) y_pred_gb = gb_best_model.predict(X_test_scaled) # Calculate performance metrics mse_rf = mean_squared_error(y_test, y_pred_rf) r2_rf = rf_best_model.score(X_test_scaled, y_test) mse_gb = mean_squared_error(y_test, y_pred_gb) r2_gb = gb_best_model.score(X_test_scaled, y_test) print(f"Random Forest - Mean Squared Error: {mse_rf}, R²: {r2_rf}") print(f"Gradient Boosting - Mean Squared Error: {mse_gb}, R²:

{r2_gb}") # Step 12: Optional: Plot feature importances for Random Forest importances = rf_best_model.feature_importances_ indices = np.argsort(importances)[::-1] features = X.columns plt.figure(figsize=(10, 6)) plt.title('Random Forest Feature Importances') plt.bar(range(X.shape[1]), importances[indices], align='center') plt.xticks(range(X.shape[1]), features[indices], rotation=90) plt.xlim([-1, X.shape[1]]) plt.show() # Final system resource check after fitting models cpu, memory = check_system_resources()
2/2

ChatGPT said:
Reasoned for 39 seconds

Below is an example script that integrates the recommendations:

- It loads the SDSSDR18_200000.csv dataset (which uses columns such as objid, ra, dec, u, g, r, i, z, specobjid, class, redshift, etc.).
- It performs basic preprocessing and feature engineering, including computing color indices.
- It also creates a rough gas-density proxy (using the r-band magnitude as a surrogate) and computes an expected SFR based on the Kennicutt–Schmidt law (Σ_SFR ∝ Σ_gas^1.4) with a normalization constant A. (You'd want to calibrate A using literature or your data.)
- It adds the computed expected SFR (and optionally the residual between observed and expected SFR if you have an observed SFR) as features.
- It uses Optuna for Bayesian optimization to tune three hyperparameters (max_depth, n_estimators, min_samples_split) for both Random Forest and Gradient Boosting.
- Finally, it trains the models and evaluates their performance.

Below is a redrafted script that implements these ideas:

python
CopyEdit

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor,
GradientBoostingRegressor
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.impute import SimpleImputer
import matplotlib.pyplot as plt
import psutil  # For monitoring system resources
import optuna  # For Bayesian optimization
from scipy.stats import randint
from numpy.polynomial.polynomial import Polynomial

# --------------------------------
# Helper: System Resource Monitor
def check_system_resources():
    import psutil
    memory = psutil.virtual_memory()
    cpu = psutil.cpu_percent(interval=1)
    print(f"CPU Usage: {cpu}%")
    print(f"Memory Usage: {memory.percent}%")
    return cpu, memory.percent

# --------------------------------
# Step 1: Load the SDSS dataset
df = pd.read_csv('SDSSDR18_200000.csv', low_memory=False)
print("Columns in SDSS DataFrame:", df.columns)
print(df.head())

# --------------------------------
# Step 2: Data Preprocessing
# Impute missing values for selected columns
imputer = SimpleImputer(strategy='mean')
cols_to_impute = ['ra', 'dec', 'redshift', 'u', 'g', 'r', 'i', 'z']
df[cols_to_impute] = imputer.fit_transform(df[cols_to_impute])

# --------------------------------
# Step 3: Feature Engineering
# Calculate color indices
df['u_g'] = df['u'] - df['g']
df['g_r'] = df['g'] - df['r']
df['r_i'] = df['r'] - df['i']
df['i_z'] = df['i'] - df['z']
```

```
# --- Integrate Kennicutt-Schmidt Law ---
# Here we create a proxy for gas density. This is a rough
approximation:
# Assume brighter (lower r magnitude) galaxies have higher gas
densities.
# (In practice, you would use direct gas measurements.)
df['gas_density'] = 10 ** (0.4 * (22 - df['r']))
# Define a normalization constant A (calibrated from literature;
adjust as needed)
A = 1e-4
# Compute expected SFR using the Kennicutt-Schmidt law (power-law
index ~1.4)
df['expected_SFR'] = A * (df['gas_density'] ** 1.4)
# If an observed SFR is available, you might compute a residual;
otherwise, expected_SFR can be used as a feature

# For this example, we use expected_SFR as an additional feature.
# Also, we'll use redshift as our target (or you could choose another
target variable).
# Here, as an example, we predict redshift using photometry (in
practice, you might predict SFR).
X = df[['ra', 'dec', 'u_g', 'g_r', 'r_i', 'i_z', 'expected_SFR']]
y = df['redshift']

# --------------------------------
# Step 4: Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# --------------------------------
# Step 5: Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# --------------------------------
# Optional: Monitor system resources before optimization
```

```
check_system_resources()

# ------------------------------
# Step 6: Bayesian Optimization with Optuna for RandomForestRegressor
def objective_rf(trial):
    max_depth = trial.suggest_int("max_depth", 5, 30)
    n_estimators = trial.suggest_int("n_estimators", 50, 500)
    min_samples_split = trial.suggest_int("min_samples_split", 2, 20)

    model = RandomForestRegressor(
        max_depth=max_depth,
        n_estimators=n_estimators,
        min_samples_split=min_samples_split,
        random_state=42,
        n_jobs=-1
    )
    # Use 3-fold cross-validation and optimize for R² score
    from sklearn.model_selection import cross_val_score
    scores = cross_val_score(model, X_train_scaled, y_train, cv=3,
scoring='r2', n_jobs=-1)
    return np.mean(scores)

study_rf = optuna.create_study(direction="maximize")
study_rf.optimize(objective_rf, n_trials=50)

print("Best RandomForest hyperparameters:", study_rf.best_params)
best_rf = RandomForestRegressor(
    **study_rf.best_params, random_state=42, n_jobs=-1
)
best_rf.fit(X_train_scaled, y_train)

# ------------------------------
# Step 7: Bayesian Optimization with Optuna for
GradientBoostingRegressor
def objective_gb(trial):
    max_depth = trial.suggest_int("max_depth", 3, 20)
    n_estimators = trial.suggest_int("n_estimators", 50, 500)
    min_samples_split = trial.suggest_int("min_samples_split", 2, 20)
```

```
    learning_rate = trial.suggest_loguniform("learning_rate", 0.01,
0.3)

    model = GradientBoostingRegressor(
        max_depth=max_depth,
        n_estimators=n_estimators,
        min_samples_split=min_samples_split,
        learning_rate=learning_rate,
        random_state=42
    )
    from sklearn.model_selection import cross_val_score
    scores = cross_val_score(model, X_train_scaled, y_train, cv=3,
scoring='r2', n_jobs=-1)
    return np.mean(scores)

study_gb = optuna.create_study(direction="maximize")
study_gb.optimize(objective_gb, n_trials=50)

print("Best GradientBoosting hyperparameters:", study_gb.best_params)
best_gb = GradientBoostingRegressor(
    **study_gb.best_params, random_state=42
)
best_gb.fit(X_train_scaled, y_train)

# ------------------------------
# Step 8: Evaluate Both Models on Test Set
y_pred_rf = best_rf.predict(X_test_scaled)
mse_rf = mean_squared_error(y_test, y_pred_rf)
r2_rf = best_rf.score(X_test_scaled, y_test)

y_pred_gb = best_gb.predict(X_test_scaled)
mse_gb = mean_squared_error(y_test, y_pred_gb)
r2_gb = best_gb.score(X_test_scaled, y_test)

print(f"Random Forest - MSE: {mse_rf}, R²: {r2_rf}")
print(f"Gradient Boosting - MSE: {mse_gb}, R²: {r2_gb}")

# ------------------------------
```

```
# Step 9: SHAP Analysis (optional)
import shap
explainer_rf = shap.TreeExplainer(best_rf)
shap_values_rf = explainer_rf.shap_values(X_test_scaled)
shap.summary_plot(shap_values_rf, X_test, feature_names=X.columns)

explainer_gb = shap.TreeExplainer(best_gb)
shap_values_gb = explainer_gb.shap_values(X_test_scaled)
shap.summary_plot(shap_values_gb, X_test, feature_names=X.columns)

# ------------------------------
# Final resource check
check_system_resources()
```

Explanation of Key Additions:

1. **Physical Model Integration (Kennicutt–Schmidt Law):**
    - We create a proxy for gas density using the r-band magnitude (a simple proxy: `10^(0.4*(22 - r))`).
    - Then we compute an expected SFR using the Kennicutt–Schmidt law with an exponent of 1.4 and a normalization constant `A`.
    - The computed `expected_SFR` is added as a new feature to the model.
2. **Bayesian Optimization with Optuna:**
    - Two objective functions are defined—one for `RandomForestRegressor` and one for `GradientBoostingRegressor`—which optimize for the R^2 score using 3-fold cross-validation.
    - Each study is run for 50 trials to find the best hyperparameters for each model.
    - The best hyperparameters are printed and used to retrain the models on the full training set.
3. **Resource Monitoring:**

    - We use a helper function (`check_system_resources()`) to monitor CPU and memory usage before and after model fitting.

4. **Standardization and Imputation:**

    - The dataset is preprocessed with mean imputation and standard scaling, ensuring consistent feature scales.

5. **SHAP Analysis:**

    - After training the best models, SHAP is used to interpret feature importance.

Next Steps:

- **Run this script** on your SDSSDR18_200000 dataset to validate the approach.

- **Calibrate the physical model:**
  Adjust the normalization constant `A` and possibly the gas density proxy if you have more precise measurements.

- **Interpret the SHAP plots** to see how the newly added physical features (like `expected_SFR`) affect the predictions.

- **Iterate and refine:**
  Depending on the performance, you might refine your feature set further or adjust the Optuna trials.

Let me know if you need further modifications or more details on any part of the script!

You said:

can you tell me what this means? (myenv) root@BigsbyGaming:/home/pmqr7# python model14.py Columns in SDSS DataFrame: Index(['objid', 'ra', 'dec', 'u', 'g', 'r', 'i', 'z', 'run', 'rerun', 'camcol', 'field', 'specobjid', 'class', 'redshift', 'plate', 'mjd', 'fiberid'], dtype='object') objid ra dec u g ... class redshift plate mjd fiberid 0 1237670961090855078 42.685546 33.497015 16.84832 15.70037 ... STAR -0.000219 7254 56625 416 1 1237670961091117213 43.015847 34.010126 17.93136 16.42230 ... STAR -0.000148 7254 56625 351 2 1237670961091641728 43.712313 35.015497 19.20811 17.95179 ... STAR 0.000536 7254 56625 754 3 1237670961086201862 36.915264 24.086372 18.78726 17.12819 ... STAR -0.000013 3241 54884 427 4 1237670961090789890 42.569470 33.447352 19.38691 17.96079 ... STAR -0.000160 7254 56625 417 [5 rows x 18 columns] CPU Usage: 0.0% Memory Usage: 21.1% [I 2025-04-06 09:41:07,065] A new study created in memory with name:

no-name-ef252aea-6bdb-44da-81a8-2a055c4c46f7 [I 2025-04-06 09:42:16,228] Trial 0 finished with value: 0.7942092445760118 and parameters: {'max_depth': 13, 'n_estimators': 191, 'min_samples_split': 11}. Best is trial 0 with value: 0.7942092445760118.

ChatGPT said:
Reasoned for 17 seconds

The output tells us several things:

1. **Data Loading and Inspection**:
    - Your SDSS dataset was successfully loaded with 18 columns.
    - The first five rows are printed, showing typical columns like `objid`, `ra`, `dec`, magnitudes (`u`, `g`, `r`, `i`, `z`), and other spectroscopic and imaging identifiers.
2. **Resource Check**:
    - The system resource check shows that CPU usage is very low (0.0%) at that moment, and memory usage is moderate (21.1%). This indicates that, at the time of reporting, your system wasn't under heavy load.
3. **Optuna Study Creation**:
    - The message `[I 2025-04-06 09:41:07,065] A new study created in memory with name: no-name-ef252aea-6bdb-44da-81a8-2a055c4c46f7` means that Optuna has started a new study for Bayesian hyperparameter optimization. This study will be used to find the best set of hyperparameters for your model.
4. **Trial Results**:
    - The line `[I 2025-04-06 09:42:16,228] Trial 0 finished with value: 0.7942092445760118 and parameters: {'max_depth': 13, 'n_estimators': 191, 'min_samples_split': 11}. Best is trial 0 with value: 0.7942092445760118.` tells us that:
        - **Trial 0** was the first evaluation in the optimization process.
        - This trial used a Random Forest configuration with a maximum tree depth of 13, 191 trees, and a minimum of 11 samples required to split an internal node.

- The objective (likely the cross-validated R^2 score) for this trial was approximately 0.794.
- Currently, Trial 0 is the best so far.

What This Means for Your Test:

- **Successful Data Load and Preprocessing**: Your dataset is correctly loaded and preprocessed.
- **Resource Availability**: Your system has adequate resources (low CPU and moderate memory usage).
- **Optuna is Working**: The Bayesian optimization process via Optuna has started and completed at least one trial successfully.
- **Promising Hyperparameter Results**: The best hyperparameters found in Trial 0 yield a cross-validation score of ~0.794, suggesting that the model is capturing a significant amount of the variance in the target (redshift in this case).

This indicates that, before any interruption, your test was operating as expected—with Optuna successfully running trials and reporting hyperparameter configurations and their performance. It now gives you a promising starting point to further optimize or use these parameters in your final model.

Would you like more details on interpreting these values or on next steps for further refinement?

You said:

the test is still in progress these are the latest responses (myenv) root@BigsbyGaming:/home/pmqr7# python model14.py Columns in SDSS DataFrame: Index(['objid', 'ra', 'dec', 'u', 'g', 'r', 'i', 'z', 'run', 'rerun', 'camcol', 'field', 'specobjid', 'class', 'redshift', 'plate', 'mjd', 'fiberid'], dtype='object') objid ra dec u g ... class redshift plate mjd fiberid 0 1237670961090855078 42.685546 33.497015 16.84832 15.70037 ... STAR -0.000219 7254 56625 416 1 1237670961091117213 43.015847 34.010126 17.93136 16.42230 ... STAR -0.000148 7254 56625 351 2 1237670961091641728 43.712313 35.015497 19.20811 17.95179 ... STAR 0.000536 7254 56625 754 3 1237670961086201862 36.915264 24.086372 18.78726 17.12819 ... STAR -0.000013 3241 54884 427 4 1237670961090789890 42.569470 33.447352 19.38691 17.96079 ... STAR -0.000160 7254 56625 417 [5 rows x 18 columns] CPU Usage: 0.0% Memory Usage: 21.1% [I 2025-04-06 09:41:07,065] A new study created in memory with name: no-name-ef252aea-6bdb-44da-81a8-2a055c4c46f7 [I 2025-04-06 09:42:16,228] Trial 0 finished with value: 0.7942092445760118 and parameters: {'max_depth': 13, 'n_estimators':

191, 'min_samples_split': 11}. Best is trial 0 with value: 0.7942092445760118. [I 2025-04-06 09:44:34,378] Trial 1 finished with value: 0.7963811275770195 and parameters: {'max_depth': 17, 'n_estimators': 300, 'min_samples_split': 6}. Best is trial 1 with value: 0.7963811275770195.

ChatGPT said:
Reasoned for 11 seconds

These messages show that Optuna is actively optimizing your model's hyperparameters. In detail:

- **Trial 0** finished with a cross-validated R^2 score of about 0.794 using a configuration with:
    - `max_depth`: 13
    - `n_estimators`: 191
    - `min_samples_split`: 11
- **Trial 1** achieved a slightly better score, approximately 0.796, using:
    - `max_depth`: 17
    - `n_estimators`: 300
    - `min_samples_split`: 6

This means that during the Bayesian optimization process, Optuna tried different parameter combinations, and so far, the configuration in Trial 1 outperforms Trial 0 in terms of the R^2 score (which is the metric being maximized). The study is iteratively exploring the parameter space and updating the "best" configuration based on the cross-validation performance.

In other words, until now the optimization is proceeding as expected, with each trial evaluating a different set of hyperparameters. The fact that the best value has improved from ~0.794 to ~0.796 indicates that the optimizer is successfully navigating the hyperparameter space to find better settings for your model.

Would you like more details on how these parameters might impact your model's behavior or suggestions on next steps in the optimization process?

You said:

can you adapt this script to target sfr, and to save the shap plot? import numpy as np import pandas as pd from sklearn.model_selection import train_test_split from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor from sklearn.preprocessing import StandardScaler from sklearn.metrics import mean_squared_error, r2_score from sklearn.impute import SimpleImputer import matplotlib.pyplot as plt import psutil # For monitoring system resources import optuna # For Bayesian optimization from scipy.stats import randint from numpy.polynomial.polynomial import Polynomial # ------------------------------ # Helper: System Resource Monitor def check_system_resources(): import psutil memory = psutil.virtual_memory() cpu = psutil.cpu_percent(interval=1) print(f"CPU Usage: {cpu}%") print(f"Memory Usage: {memory.percent}%") return cpu, memory.percent # ------------------------------ # Step 1: Load the SDSS dataset df = pd.read_csv('SDSSDR18_200000.csv', low_memory=False) print("Columns in SDSS DataFrame:", df.columns) print(df.head()) # ------------------------------ # Step 2: Data Preprocessing # Impute missing values for selected columns imputer = SimpleImputer(strategy='mean') cols_to_impute = ['ra', 'dec', 'redshift', 'u', 'g', 'r', 'i', 'z'] df[cols_to_impute] = imputer.fit_transform(df[cols_to_impute]) # ------------------------------ # Step 3: Feature Engineering # Calculate color indices df['u_g'] = df['u'] - df['g'] df['g_r'] = df['g'] - df['r'] df['r_i'] = df['r'] - df['i'] df['i_z'] = df['i'] - df['z'] # --- Integrate Kennicutt-Schmidt Law --- # Here we create a proxy for gas density. This is a rough approximation: # Assume brighter (lower r magnitude) galaxies have higher gas densities. # (In practice, you would use direct gas measurements.) df['gas_density'] = 10 ** (0.4 * (22 - df['r'])) # Define a normalization constant A (calibrated from literature; adjust as needed) A = 1e-4 # Compute expected SFR using the Kennicutt-Schmidt law (power-law index ~1.4) df['expected_SFR'] = A * (df['gas_density'] ** 1.4) # If an observed SFR is available, you might compute a residual; otherwise, expected_SFR can be used as a feature # For this example, we use expected_SFR as an additional feature. # Also, we'll use redshift as our target (or you could choose another target variable). # Here, as an example, we predict redshift using photometry (in practice, you might predict SFR). X = df[['ra', 'dec', 'u_g', 'g_r', 'r_i', 'i_z', 'expected_SFR']] y = df['redshift'] # ------------------------------ # Step 4: Split data into training and testing sets X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42) # ------------------------------ # Step 5: Standardize the features scaler = StandardScaler() X_train_scaled = scaler.fit_transform(X_train) X_test_scaled = scaler.transform(X_test) # ------------------------------ # Optional: Monitor system resources before optimization check_system_resources() # ------------------------------ # Step 6: Bayesian Optimization with Optuna for RandomForestRegressor def objective_rf(trial): max_depth = trial.suggest_int("max_depth", 5, 30) n_estimators = trial.suggest_int("n_estimators", 50, 500) min_samples_split = trial.suggest_int("min_samples_split", 2, 20) model = RandomForestRegressor(max_depth=max_depth, n_estimators=n_estimators, min_samples_split=min_samples_split, random_state=42, n_jobs=-1) # Use 3-fold cross-validation and optimize for R² score from sklearn.model_selection import cross_val_score scores = cross_val_score(model, X_train_scaled, y_train, cv=3, scoring='r2', n_jobs=-1) return np.mean(scores) study_rf = optuna.create_study(direction="maximize") study_rf.optimize(objective_rf, n_trials=50) print("Best RandomForest hyperparameters:", study_rf.best_params) best_rf = RandomForestRegressor(**study_rf.best_params, random_state=42, n_jobs=-1) best_rf.fit(X_train_scaled, y_train) # ------------------------------ #

Step 7: Bayesian Optimization with Optuna for GradientBoostingRegressor def objective_gb(trial): max_depth = trial.suggest_int("max_depth", 3, 20) n_estimators = trial.suggest_int("n_estimators", 50, 500) min_samples_split = trial.suggest_int("min_samples_split", 2, 20) learning_rate = trial.suggest_loguniform("learning_rate", 0.01, 0.3) model = GradientBoostingRegressor(max_depth=max_depth, n_estimators=n_estimators, min_samples_split=min_samples_split, learning_rate=learning_rate, random_state=42) from sklearn.model_selection import cross_val_score scores = cross_val_score(model, X_train_scaled, y_train, cv=3, scoring='r2', n_jobs=-1) return np.mean(scores) study_gb = optuna.create_study(direction="maximize") study_gb.optimize(objective_gb, n_trials=50) print("Best GradientBoosting hyperparameters:", study_gb.best_params) best_gb = GradientBoostingRegressor(**study_gb.best_params, random_state=42) best_gb.fit(X_train_scaled, y_train) # ---------- # Step 8: Evaluate Both Models on Test Set y_pred_rf = best_rf.predict(X_test_scaled) mse_rf = mean_squared_error(y_test, y_pred_rf) r2_rf = best_rf.score(X_test_scaled, y_test) y_pred_gb = best_gb.predict(X_test_scaled) mse_gb = mean_squared_error(y_test, y_pred_gb) r2_gb = best_gb.score(X_test_scaled, y_test) print(f"Random Forest - MSE: {mse_rf}, R²: {r2_rf}") print(f"Gradient Boosting - MSE: {mse_gb}, R²: {r2_gb}") # ---------- # Step 9: SHAP Analysis (optional) import shap explainer_rf = shap.TreeExplainer(best_rf) shap_values_rf = explainer_rf.shap_values(X_test_scaled) shap.summary_plot(shap_values_rf, X_test, feature_names=X.columns) explainer_gb = shap.TreeExplainer(best_gb) shap_values_gb = explainer_gb.shap_values(X_test_scaled) shap.summary_plot(shap_values_gb, X_test, feature_names=X.columns) # ---------- # Final resource check check_system_resources()

ChatGPT said:

To adapt your script to predict the star formation rate (SFR) and save the SHAP plots as image files, you'll need to make the following modifications:

1. **Target Variable**: Ensure your dataset includes an 'sfr' column representing the star formation rate. If it doesn't, you'll need to obtain or calculate this data.

Update the Target Variable: Change the target variable from 'redshift' to 'sfr':

python
CopyEdit
```
y = df['sfr']
```

2.

Save SHAP Plots: To save the SHAP summary plots as image files, set `show=False` in the `shap.summary_plot()` function and then use Matplotlib's `savefig()` function to save the

plot:YouTube

python
CopyEdit

```
import shap
import matplotlib.pyplot as plt

# For Random Forest
explainer_rf = shap.TreeExplainer(best_rf)
shap_values_rf = explainer_rf.shap_values(X_test_scaled)
shap.summary_plot(shap_values_rf, X_test, feature_names=X.columns,
show=False)
plt.savefig('shap_summary_rf.png')  # Save the plot as a PNG file
plt.close()  # Close the plot to free up memory

# For Gradient Boosting
explainer_gb = shap.TreeExplainer(best_gb)
shap_values_gb = explainer_gb.shap_values(X_test_scaled)
shap.summary_plot(shap_values_gb, X_test, feature_names=X.columns,
show=False)
plt.savefig('shap_summary_gb.png')  # Save the plot as a PNG file
plt.close()  # Close the plot to free up memory
```

3.

This approach ensures that the SHAP plots are saved as image files without being displayed during script execution.

Ensure that your dataset includes the 'sfr' column and that the necessary libraries (`shap`, `matplotlib`) are installed in your environment.

Sources

You said:

Canvas can you adapt this script to target sfr, (as it already calculates for sfr) and to save the shap plot? import numpy as np import pandas as pd from sklearn.model_selection import train_test_split from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor from sklearn.preprocessing import StandardScaler from

```
sklearn.metrics import mean_squared_error, r2_score from sklearn.impute import SimpleImputer import matplotlib.pyplot as plt import psutil # For monitoring system resources import optuna # For Bayesian optimization from scipy.stats import randint from numpy.polynomial.polynomial import Polynomial # ------------------------------ # Helper: System Resource Monitor def check_system_resources(): import psutil memory = psutil.virtual_memory() cpu = psutil.cpu_percent(interval=1) print(f"CPU Usage: {cpu}%") print(f"Memory Usage: {memory.percent}%") return cpu, memory.percent # ------------------------------ # Step 1: Load the SDSS dataset df = pd.read_csv('SDSSDR18_200000.csv', low_memory=False) print("Columns in SDSS DataFrame:", df.columns) print(df.head()) # ------------------------------ # Step 2: Data Preprocessing # Impute missing values for selected columns imputer = SimpleImputer(strategy='mean') cols_to_impute = ['ra', 'dec', 'redshift', 'u', 'g', 'r', 'i', 'z'] df[cols_to_impute] = imputer.fit_transform(df[cols_to_impute]) # ------------------------------ # Step 3: Feature Engineering # Calculate color indices df['u_g'] = df['u'] - df['g'] df['g_r'] = df['g'] - df['r'] df['r_i'] = df['r'] - df['i'] df['i_z'] = df['i'] - df['z'] # --- Integrate Kennicutt-Schmidt Law --- # Here we create a proxy for gas density. This is a rough approximation: # Assume brighter (lower r magnitude) galaxies have higher gas densities. # (In practice, you would use direct gas measurements.) df['gas_density'] = 10 ** (0.4 * (22 - df['r'])) # Define a normalization constant A (calibrated from literature; adjust as needed) A = 1e-4 # Compute expected SFR using the Kennicutt-Schmidt law (power-law index ~1.4) df['expected_SFR'] = A * (df['gas_density'] ** 1.4) # If an observed SFR is available, you might compute a residual; otherwise, expected_SFR can be used as a feature # For this example, we use expected_SFR as an additional feature. # Also, we'll use redshift as our target (or you could choose another target variable). # Here, as an example, we predict redshift using photometry (in practice, you might predict SFR). X = df[['ra', 'dec', 'u_g', 'g_r', 'r_i', 'i_z', 'expected_SFR']] y = df['redshift'] # ------------------------------ # Step 4: Split data into training and testing sets X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42) # ------------------------------ # Step 5: Standardize the features scaler = StandardScaler() X_train_scaled = scaler.fit_transform(X_train) X_test_scaled = scaler.transform(X_test) # ------------------------------ # Optional: Monitor system resources before optimization check_system_resources() # ------------------------------ # Step 6: Bayesian Optimization with Optuna for RandomForestRegressor def objective_rf(trial): max_depth = trial.suggest_int("max_depth", 5, 30) n_estimators = trial.suggest_int("n_estimators", 50, 500) min_samples_split = trial.suggest_int("min_samples_split", 2, 20) model = RandomForestRegressor( max_depth=max_depth, n_estimators=n_estimators, min_samples_split=min_samples_split, random_state=42, n_jobs=-1 ) # Use 3-fold cross-validation and optimize for R² score from sklearn.model_selection import cross_val_score scores = cross_val_score(model, X_train_scaled, y_train, cv=3, scoring='r2', n_jobs=-1) return np.mean(scores) study_rf = optuna.create_study(direction="maximize") study_rf.optimize(objective_rf, n_trials=50) print("Best RandomForest hyperparameters:", study_rf.best_params) best_rf = RandomForestRegressor( **study_rf.best_params, random_state=42, n_jobs=-1 ) best_rf.fit(X_train_scaled, y_train) # ------------------------------ # Step 7: Bayesian Optimization with Optuna for GradientBoostingRegressor def objective_gb(trial): max_depth = trial.suggest_int("max_depth", 3, 20) n_estimators = trial.suggest_int("n_estimators", 50, 500) min_samples_split =
```

trial.suggest_int("min_samples_split", 2, 20) learning_rate = trial.suggest_loguniform("learning_rate", 0.01, 0.3) model = GradientBoostingRegressor(max_depth=max_depth, n_estimators=n_estimators, min_samples_split=min_samples_split, learning_rate=learning_rate, random_state=42) from sklearn.model_selection import cross_val_score scores = cross_val_score(model, X_train_scaled, y_train, cv=3, scoring='r2', n_jobs=-1) return np.mean(scores) study_gb = optuna.create_study(direction="maximize") study_gb.optimize(objective_gb, n_trials=50) print("Best GradientBoosting hyperparameters:", study_gb.best_params) best_gb = GradientBoostingRegressor(**study_gb.best_params, random_state=42) best_gb.fit(X_train_scaled, y_train) # ---------- # Step 8: Evaluate Both Models on Test Set y_pred_rf = best_rf.predict(X_test_scaled) mse_rf = mean_squared_error(y_test, y_pred_rf) r2_rf = best_rf.score(X_test_scaled, y_test) y_pred_gb = best_gb.predict(X_test_scaled) mse_gb = mean_squared_error(y_test, y_pred_gb) r2_gb = best_gb.score(X_test_scaled, y_test) print(f"Random Forest - MSE: {mse_rf}, R²: {r2_rf}") print(f"Gradient Boosting - MSE: {mse_gb}, R²: {r2_gb}") # ---------- # Step 9: SHAP Analysis (optional) import shap explainer_rf = shap.TreeExplainer(best_rf) shap_values_rf = explainer_rf.shap_values(X_test_scaled) shap.summary_plot(shap_values_rf, X_test, feature_names=X.columns) explainer_gb = shap.TreeExplainer(best_gb) shap_values_gb = explainer_gb.shap_values(X_test_scaled) shap.summary_plot(shap_values_gb, X_test, feature_names=X.columns) # ---------- # Final resource check check_system_resources()

ChatGPT said:

Sfr Shap Analysis

Sfr Shap Analysis

Done! Your full SHAP-based SFR prediction pipeline with Optuna optimization has been saved to the canvas as `sfr_shap_analysis`. Let me know if you'd like to:

- Add PySR symbolic regression next
- Join SDSSDR18 with the GAMA dataset
- Interpret the SHAP results together
- Generate SHAP dependence plots
- Or dive deeper into the Birch and Swinnerton-Dyer analogies for SFR!

Addition of PySR

PySR (Symbolic Regression) into your pipeline to extract interpretable mathematical expressions that predict **SFR** from your features.

We'll add this in the next steps of your script after SHAP analysis.

✅ PySR Setup (Step 10):

Here's how we'll proceed:

1. **Install and import PySR**
2. **Format the dataset** (`X_train`, `y_train`) for symbolic regression
3. **Fit a PySRRegressor model**
4. **Display the symbolic expressions**
5. **Save the model and expressions**

✅ Updated Code Block to Add:

python
CopyEdit

```
# ------------------------------
# Step 10: Symbolic Regression with PySR
from pysr import PySRRegressor

# Optional: reduce dimensionality or add more engineered features
# You may already have symbolic candidate features (like u_g, g_r,
etc.)

symbolic_model = PySRRegressor(
    model_selection="best",  # Select best model according to loss +
complexity
    niterations=100,         # Increase for better results
    population_size=1000,
```

```
    binary_operators=["+", "-", "*", "/"],
    unary_operators=["sqrt", "log", "exp"],
    loss="loss(x, y) = (x - y)^2",
    maxsize=20,
    verbosity=1,
    random_state=42,
    temp_equation_file=True  # Save intermediate results
)

# Fit PySR on raw (unscaled) data for interpretability
symbolic_model.fit(X_train, y_train)

# Display the discovered symbolic equations
print("Best symbolic models:\n", symbolic_model)

# Predict and evaluate
y_pysr_pred = symbolic_model.predict(X_test)
r2_pysr = r2_score(y_test, y_pysr_pred)
print(f"PySR R² score: {r2_pysr:.4f}")

# Save model results
symbolic_model.save("pysr_model.pkl")
symbolic_model.equations_.to_csv("pysr_equations.csv", index=False)
```

✅ Additional Suggestions:

- If you want to **link symbolic terms back to physical quantities**, we can rename features in `X.columns` (e.g., `'g_r' → 'color_gr'`, `'expected_SFR' → 'SFR_KS'`).
- You can increase `niterations` to 500+ for higher-quality equations.
- Add `extra_sympy_mappings={"log10": "log10"}` if you want to explicitly work in base-10 log space (like log_Mass_gas or log_SFR).

```
import numpy as np
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.impute import SimpleImputer
import matplotlib.pyplot as plt
import psutil
import optuna
from pysr import PySRRegressor
import shap

# --------------------------------
# Helper: System Resource Monitor
def check_system_resources():
    memory = psutil.virtual_memory()
    cpu = psutil.cpu_percent(interval=1)
    print(f"CPU Usage: {cpu}%")
    print(f"Memory Usage: {memory.percent}%")
    return cpu, memory.percent

# --------------------------------
# Step 1: Load the SDSS dataset
df = pd.read_csv('SDSSDR18_200000.csv', low_memory=False)
print("Columns in SDSS DataFrame:", df.columns)
print(df.head())

# --------------------------------
# Step 2: Data Preprocessing
imputer = SimpleImputer(strategy='mean')
cols_to_impute = ['ra', 'dec', 'redshift', 'u', 'g', 'r', 'i', 'z']
df[cols_to_impute] = imputer.fit_transform(df[cols_to_impute])

# --------------------------------
# Step 3: Feature Engineering
# Calculate color indices
df['u_g'] = df['u'] - df['g']
df['g_r'] = df['g'] - df['r']
df['r_i'] = df['r'] - df['i']
df['i_z'] = df['i'] - df['z']

# Kennicutt-Schmidt Law proxy
df['gas_density'] = 10 ** (0.4 * (22 - df['r']))
A = 1e-4
df['expected_SFR'] = A * (df['gas_density'] ** 1.4)
```

```
# Define features and target
X = df[['ra', 'dec', 'u_g', 'g_r', 'r_i', 'i_z', 'expected_SFR']]
y = df['expected_SFR']  # Target is SFR

# ------------------------------
# Step 4: Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# ------------------------------
# Step 5: Standardize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# ------------------------------
# Step 6: Random Forest - Optuna
check_system_resources()

def objective_rf(trial):
    max_depth = trial.suggest_int("max_depth", 5, 30)
    n_estimators = trial.suggest_int("n_estimators", 50, 500)
    min_samples_split = trial.suggest_int("min_samples_split", 2, 20)
    model = RandomForestRegressor(max_depth=max_depth, n_estimators=n_estimators,
min_samples_split=min_samples_split, random_state=42, n_jobs=-1)
    from sklearn.model_selection import cross_val_score
    scores = cross_val_score(model, X_train_scaled, y_train, cv=3, scoring='r2', n_jobs=-1)
    return np.mean(scores)

study_rf = optuna.create_study(direction="maximize")
study_rf.optimize(objective_rf, n_trials=50)

best_rf = RandomForestRegressor(**study_rf.best_params, random_state=42, n_jobs=-1)
best_rf.fit(X_train_scaled, y_train)

# ------------------------------
# Step 7: Gradient Boosting - Optuna
def objective_gb(trial):
    max_depth = trial.suggest_int("max_depth", 3, 20)
    n_estimators = trial.suggest_int("n_estimators", 50, 500)
    min_samples_split = trial.suggest_int("min_samples_split", 2, 20)
    learning_rate = trial.suggest_loguniform("learning_rate", 0.01, 0.3)
    model = GradientBoostingRegressor(max_depth=max_depth, n_estimators=n_estimators,
min_samples_split=min_samples_split, learning_rate=learning_rate, random_state=42)
```

```
    from sklearn.model_selection import cross_val_score
    scores = cross_val_score(model, X_train_scaled, y_train, cv=3, scoring='r2', n_jobs=-1)
    return np.mean(scores)

study_gb = optuna.create_study(direction="maximize")
study_gb.optimize(objective_gb, n_trials=50)

best_gb = GradientBoostingRegressor(**study_gb.best_params, random_state=42)
best_gb.fit(X_train_scaled, y_train)

# --------------------------------
# Step 8: Evaluate Models
y_pred_rf = best_rf.predict(X_test_scaled)
mse_rf = mean_squared_error(y_test, y_pred_rf)
r2_rf = r2_score(y_test, y_pred_rf)

y_pred_gb = best_gb.predict(X_test_scaled)
mse_gb = mean_squared_error(y_test, y_pred_gb)
r2_gb = r2_score(y_test, y_pred_gb)

print(f"Random Forest - MSE: {mse_rf}, R²: {r2_rf}")
print(f"Gradient Boosting - MSE: {mse_gb}, R²: {r2_gb}")

# --------------------------------
# Step 9: SHAP Analysis
explainer_rf = shap.Explainer(best_rf)
shap_values_rf = explainer_rf(X_test_scaled)
shap.summary_plot(shap_values_rf, X_test, feature_names=X.columns, show=False)
plt.title("SHAP Summary - Random Forest")
plt.savefig("shap_summary_rf.png")

explainer_gb = shap.Explainer(best_gb)
shap_values_gb = explainer_gb(X_test_scaled)
shap.summary_plot(shap_values_gb, X_test, feature_names=X.columns, show=False)
plt.title("SHAP Summary - Gradient Boosting")
plt.savefig("shap_summary_gb.png")

# --------------------------------
# Step 10: Symbolic Regression with PySR
symbolic_model = PySRRegressor(
    model_selection="best",
    niterations=100,
    population_size=1000,
    binary_operators=["+", "-", "*", "/"],
```

```
    unary_operators=["sqrt", "log", "exp"],
    loss="loss(x, y) = (x - y)^2",
    maxsize=20,
    verbosity=1,
    random_state=42
)

symbolic_model.fit(X_train, y_train)
print("Best symbolic models:\n", symbolic_model)

# Predict and evaluate
y_pysr_pred = symbolic_model.predict(X_test)
r2_pysr = r2_score(y_test, y_pysr_pred)
print(f"PySR R² score: {r2_pysr:.4f}")

# Save model and expressions
symbolic_model.save("pysr_model.pkl")
symbolic_model.equations_.to_csv("pysr_equations.csv", index=False)

# --------------------------------
# Final system resource check
check_system_resources()
```